

## genAnaLex + YAPar

Toolkit en Go para construir y ejecutar analizadores léxicos con **YALex** y analizadores sintácticos con **YAPar**. El proyecto permite compilar especificaciones `.yal`, tokenizar entradas, construir tablas sintácticas y comparar distintos métodos de parsing.

**Repositorio:** <https://github.com/Jonialen/proyecto-compis>

### Estado actual

| Componente           | Estado                                                                                                |
|----------------------|-------------------------------------------------------------------------------------------------------|
| YALex                | Funcional: compila <code>.yal</code> , tokeniza archivos y genera lexer standalone.                   |
| YAPar LL(1)          | Backend ejecutable disponible.                                                                        |
| YAPar SLR(1)         | Backend ejecutable disponible.                                                                        |
| YAPar LALR           | Backend ejecutable disponible.                                                                        |
| YAPar LR(0) / LR(1)  | Existen como opciones comparables, pero <b>no</b> como backends ejecutables.                          |
| Parser standalone    | Disponible para el camino basado en tablas LR/SLR; no debe asumirse universal para todos los métodos. |
| IDE/interfaz gráfica | Disponible como app web servida desde <code>cmd/ide</code> .                                          |

La referencia técnica principal está en `docs/documentacion_tecnica.md`.

### Requisitos

- Go 1.26.1+
- Graphviz opcional para renderizar archivos DOT generados

### Estructura del proyecto

```
cmd/  
  ide/      # Servidor web para la interfaz gráfica  
  yalex/    # CLI del lexer  
  yapar/    # CLI del parser y comparador  
  
internal/  
  dfa/      # Árbol sintáctico, followpos, DFA y minimización  
  generator/ # Generadores standalone  
  lexbuild/  # Pipeline reutilizable de YALex  
  lexer/     # Simulador léxico  
  regex/     # Normalización y postfix para regex
```

```
shared/      # Tipos compartidos
yalex/       # Parser y expansor de YALex
yapar/       # Gramática, tablas, backends, visualización y comparación

docs/
testdata/
web/
  # Interfaz gráfica Furlantran
```

## Uso rápido

### 0. Ejecutar la interfaz gráfica

Levantar la IDE web local:

```
go run ./cmd/ide
```

Luego abrir:

<http://localhost:8080>

Desde la interfaz se puede:

- cargar o editar archivos `.yal`, `.yalp` y entradas de prueba;
- seleccionar métodos `LL(1)`, `SLR(1)` y `LALR`;
- ver tokens, tablas, resumen de aceptación y autómatas cuando el método lo permite.

### 1. Ejecutar YALex

Tokenizar una entrada con una especificación `.yal`:

```
go run ./cmd/yalex -yal testdata/lexer.yal -src testdata/test.lisp
```

Generar un lexer standalone:

```
go run ./cmd/yalex -yal testdata/lexer.yal -out lexer_gen.go
```

Generar el árbol en formato DOT:

```
go run ./cmd/yalex -yal testdata/lexer.yal -tree
```

Notas importantes:

- `-yal` es obligatorio.
- Debe usarse al menos una salida: `-src`, `-out` o `-tree`.
- `-tree` escribe `tree.dot` en el directorio actual.

### 2. Ejecutar YAPar

Fixtures disponibles para probar el parser:

| Archivo                        | Uso                        |
|--------------------------------|----------------------------|
| testdata/yapar/arithmetic.yalp | Especificación del parser. |
| testdata/yapar/arithmetic.yal  | Especificación del lexer.  |
| testdata/yapar/valid.expr      | Entrada válida.            |
| testdata/yapar/invalid.expr    | Entrada inválida.          |

Construir el parser con el método por defecto:

```
go run ./cmd/yapar -yalp testdata/yapar/arithmetic.yalp
```

Mostrar tabla SLR en texto:

```
go run ./cmd/yapar -yalp testdata/yapar/arithmetic.yalp -method slr -table
```

Exportar autómata LALR en DOT:

```
go run ./cmd/yapar -yalp testdata/yapar/arithmetic.yalp -method lalr -format dot
```

Exportar tabla LL(1) en JSON:

```
go run ./cmd/yapar -yalp testdata/yapar/arithmetic.yalp -method ll1 -format json
```

Tokenizar y parsear una entrada real:

```
go run ./cmd/yapar -yalp testdata/yapar/arithmetic.yalp -yal testdata/yapar/arithmetic.yal
```

## Comparar métodos de parsing

YAPar puede comparar ll1, lr0, slr, lr1 y lalr desde la CLI:

```
go run ./cmd/yapar -yalp testdata/yapar/arithmetic.yalp -compare
```

```
go run ./cmd/yapar -yalp testdata/yapar/arithmetic.yalp -compare -format json
```

```
go run ./cmd/yapar -yalp testdata/yapar/arithmetic.yalp -yal testdata/yapar/arithmetic.yal
```

Si ves rutas como `mi_parser.yalp` o `entrada.txt` en notas históricas, trátalas como placeholders. Los comandos anteriores usan fixtures reales del repositorio.

## Flags principales de cmd/yapar

| Flag     | Descripción                        | Notas                                           |
|----------|------------------------------------|-------------------------------------------------|
| -yalp    | Ruta del parser .yalp.             | Obligatorio.                                    |
| -method  | Método solicitado.                 | ll1, lr0, slr, lr1, lalr.                       |
| -compare | Ejecuta comparación entre métodos. | Ignora -method; no permite -out ni -format dot. |
| -format  | Formato de visualización.          | text, json, dot.                                |

| Flag                | Descripción                           | Notas                                                                                            |
|---------------------|---------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>-table</code> | Atajo para tabla en texto.            | Equivale a salida <code>text</code> .                                                            |
| <code>-yal</code>   | Lexer <code>.yal</code> .             | Debe usarse junto con <code>-src</code> .                                                        |
| <code>-src</code>   | Archivo fuente a tokenizar y parsear. | Debe usarse junto con <code>-yal</code> .                                                        |
| <code>-out</code>   | Genera parser standalone.             | No usar con <code>-compare</code> ; <code>lr1</code> y <code>lalr</code> no soportan este flujo. |

## Parser standalone

El parser generado con `-out` consume tokens en JSON:

```
go run parser_gen.go -tokens tokens.json
```

Formato esperado:

```
[
  { "type": "ID", "lexeme": "x", "line": 1 }
]
```

## Limitaciones conocidas

- `lr0` y `lr1` aparecen en la comparación y en `-method`, pero hoy son placeholders no implementados.
- `-format dot` no aplica al modo comparativo y no todos los métodos tienen autómata exportable.
- El parser standalone no garantiza soporte completo para todos los métodos.
- La interfaz gráfica depende del servidor local `cmd/ide` y de assets web en `web/`.

## Más documentación

- `docs/documentacion_tecnica.md`: arquitectura, estado real y detalle técnico.
- `docs/parte2/`: planeación y backlogs históricos.